

Foundations of Language Modeling

Chapter 3: Step-by-Step Implementation

1 Sanity Checking the Initialization

Before writing the training loop, it is crucial to verify that the model is functioning correctly at initialization. Because the network's weights (the embedding matrix in our bigram model) are initialized randomly, the model should ideally have no prior bias toward any specific token.

Theorem 1.1 (Expected Initial Loss). *Let V be the vocabulary size. If a language model's parameters are initialized such that it outputs a perfectly uniform probability distribution over the vocabulary for any given context, then $p_i = \frac{1}{V}$ for all $i \in \{1, \dots, V\}$. The expected cross-entropy loss $\mathcal{L}_0$ at initialization is:*

$$\mathcal{L}_0 = -\log\left(\frac{1}{V}\right) = \log(V) \quad (1)$$

Intuition: If $V = 65$, then $\mathcal{L}_0 = -\log(1/65) \approx 4.174$. If we run a forward pass on a freshly initialized model and compute the loss, it should be close to this theoretical value. If the loss is significantly higher (e.g., 10.5), it indicates the initial weights are too "peaky" (overconfident in incorrect answers), and the initialization scheme might need adjustment.

2 The Optimization Algorithm

To improve the model's predictions, we must minimize the cross-entropy loss with respect to the model's parameters (denoted by θ).

Definition 2.1 (Gradient Descent Step). *Let $\mathcal{L}(\theta)$ be the scalar loss evaluated on a batch of data, and $\eta > 0$ be the learning rate. The gradient of the loss with respect to the parameters is $\nabla_{\theta}\mathcal{L}$. The simplest parameter update rule is:*

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (2)$$

In practice, pure Stochastic Gradient Descent (SGD) can be slow or get stuck in local minima. Instead, we use **AdamW**, an advanced optimizer that incorporates momentum (keeping track of a moving average of past gradients) and adaptive learning rates for each parameter, along with weight decay for regularization.

3 The PyTorch Training Loop

Training a neural network in PyTorch follows a highly standardized sequence of operations for each batch of data:

1. **Sample:** Fetch a mini-batch of inputs $\mathbf{X}$ and targets $\mathbf{Y}$.

2. **Forward Pass:** Pass $\mathbf{X}$ through the model to compute the logits and the loss $\mathcal{L}$ against $\mathbf{Y}$.
3. **Zero Gradients:** Clear the gradients from the previous iteration using `optimizer.zero_grad()`. (PyTorch accumulates gradients by default).
4. **Backward Pass:** Compute $\nabla_{\theta}\mathcal{L}$ using backpropagation by calling `loss.backward()`.
5. **Update:** Adjust the parameters by calling `optimizer.step()`.

4 Robust Evaluation and Loss Estimation

Because we train on randomly sampled mini-batches, the loss at any single iteration is a "noisy" estimate of the true loss over the entire dataset. To obtain a reliable metric of our model's performance, we must temporarily pause training, evaluate the loss over multiple batches from both the training and validation sets, and average the results.

Remark 4.1 (Evaluation Modes). *During this evaluation phase, we must instruct PyTorch to disable certain training-specific behaviors:*

- *`torch.no_grad()`: A context manager that disables gradient tracking. This drastically reduces memory consumption and speeds up computation since PyTorch no longer needs to build the computational graph.*
- *`model.eval()`: Sets the model to evaluation mode. While not strictly necessary for the pure Bigram model, it is mandatory when using layers like `Dropout` or `BatchNorm`, which behave differently during training and inference. We revert to `model.train()` once evaluation is complete.*

5 Worked Examples and Solved Exercises

5.1 Mathematical Exercise: Evaluating the Loss Baseline

Problem: You initialize a bigram model on a dataset with a vocabulary of $V = 10,000$ tokens.

1. Calculate the theoretically ideal initial cross-entropy loss.
2. Suppose the actual initial loss on the first batch is computed as 15.2. Calculate the average probability p_{target} the model assigned to the correct target tokens in this batch.

Solution: Part 1: The ideal initial loss $\mathcal{L}_0$ assumes a uniform distribution $p_i = \frac{1}{10000} = 10^{-4}$.

$$\mathcal{L}_0 = -\log(10^{-4}) = 4\log(10) \approx 4 \times 2.302 = 9.210 \quad (3)$$

Part 2: We know that the cross-entropy loss $\mathcal{L}$ is defined as $-\log(p_{target})$.

$$\begin{aligned}\mathcal{L} &= 15.2 \\ -\log(p_{target}) &= 15.2 \\ p_{target} &= \exp(-15.2) \approx 2.5 \times 10^{-7}\end{aligned}$$

The average probability assigned to the correct token is 2.5×10^{-7} , which is significantly worse than a random guess (10^{-4}). This implies the model is aggressively predicting the *wrong* tokens at initialization.

5.2 Coding Example: The Estimation Function

Problem: Write the Python function `estimate_loss()` that calculates the average loss over `eval_iters` batches for both the 'train' and 'val' splits.

Solution:

```
1  @torch.no_grad() # Tell PyTorch we will not call .backward() here
2  def estimate_loss():
3      out = {}
4      model.eval() # Switch model to evaluation phase
5
6      for split in ['train', 'val']:
7          losses = torch.zeros(eval_iters)
8          for k in range(eval_iters):
9              X, Y = get_batch(split) # Fetch data
10             logits, loss = model(X, Y) # Forward pass
11             losses[k] = loss.item() # Store the scalar loss
12             out[split] = losses.mean().item() # Calculate the mean
13
14         model.train() # Revert to training phase
15         return out
16
```

5.3 Coding Exercise: The Complete Training Loop

Problem: Using the PyTorch AdamW optimizer, implement the standard training loop. The loop should run for `max_iters` iterations. Every `eval_interval` iterations, it should call `estimate_loss()` and print the results.

Solution:

```

1      # 1. Instantiate the optimizer
2      optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
3
4      # 2. Begin the training loop
5      for iter in range(max_iters):
6
7          # Every once in a while evaluate the loss on train and val sets
8          if iter % eval_interval == 0:
9              losses = estimate_loss()
10             print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")
11
12             # Sample a batch of data
13             xb, yb = get_batch('train')
14
15             # Evaluate the loss (Forward pass)
16             logits, loss = model(xb, yb)
17
18             # Optimizer zero grad (Clear previous gradients)
19             optimizer.zero_grad(set_to_none=True)
20
21             # Backward pass (Calculate gradients)
22             loss.backward()
23
24             # Step (Update parameters)
25             optimizer.step()
26

```